

1. 论文名称：It's About Time: Incorporating Temporality in Retrieval Augmented Language Models

Abstract

- 研究问题：如何确保信息检索系统能提供最相关且最新的信息，特别是在网页内容存在多个版本且时间点不同的情况下。
- 现有挑战：大型语言模型（LLMs）常用于问答工具，但会产生信息幻觉，并且处理信息时间维度上存在困难。
- 现有技术：增强型检索器语言模型（RALMs），旨在减少LLM的信息幻觉，但仍不能正确处理时间查询。
- 提出解决方案：TempRALM，一个考虑时间因素的RALM，能同时处理语义和时间上的相关性。
- 成果展示：TempRALM在性能上相较于基准RALM模型提升高达74%，且不需要额外的计算密集型元素。

1 Introduction

背景：

- 互联网是不断扩展的实时世界知识库，文本文件占据了其内容的显著部分。
- 信息随时间变化而变化，文档不断更新或新增，导致不同时间框架的多个版本信息共存。

挑战：

- 确保用户随时获得最相关和最新知识是信息检索的主要挑战。
- 大型语言模型（LLMs）在问答工具中的使用增加了这一挑战，因为它们会生成幻觉式信息并且处理静态快照数据。

现有技术：

- 增强型检索器语言模型（RALMs）使用外部文档语料库作为知识源，减少了LLMs的幻觉，生成更具体、事实性的回应。
- RALMs存储在文档索引中，可以更新以反映当前文档的版本，但在多版本和持续更新的环境中面临挑战。

现有技术的限制：

- 标准RALM模型无法处理涉及最新数据的查询，因为它们不考虑时间元数据。
- 例如，询问“谁赢得了温布尔登男子冠军？”的查询可能因时间戳的不匹配而无法检索到相关文档。
- 先前的解决方案尝试通过预训练模型和时间上下文处理时间问题，但这些方法成本高且静态。

提出的解决方案：

- TempRALM，一种新模型，扩展了RALM的文档检索和排名算法，同时考虑语义和时间相关性。
- TempRALM在计算效率方面实现了高达74%的性能提升，不需要预训练、重计算或替换文档索引。

2 Related work

大型语言模型（LLMs）：

- LLMs和交互式接口如ChatGPT已经开始改变我们发现和理解实时信息的方式。
- 这些系统可能会产生幻觉，并给出虽然听起来很自信但实际上错误的答案，这限制了它们在知识密集型任务上的应用，例如问答、事实核查等。

检索增强型语言模型（RALMs）：

- RALMs通过添加文档数据库作为知识源来缓解上述问题。
- RALMs在问题回答和事实核查等知识密集型任务上显著提升了LLMs的性能。
- RALMs使用维基百科快照来构建文档索引，并通过检索模型来获取查询相关的文档集，然后LLM利用这些文档来生成回答。
- RALMs训练过程不需要复杂的数据标注。

Atlas模型：

- 为减少大数据集训练需求，提出了Atlas，一个使用少量样本学习的预训练模型。
- Atlas包括基于BERT的预训练检索器和T5语言模型，后者用于执行序列到序列的任务，如摘要、翻译和问答。
- Atlas模型和T5在大型数据集上进行了联合微调，并且已证明它可以用很少的样本来适应不同的任务。

时间敏感问题：

- 时间敏感的问题是对LLMs和RALMs都具有挑战性的问题类型，因为它们的答案会随时间变化，如“美国总统是谁？”。
- 先前的研究探索了这些问题对LLMs的挑战，并表明预训练模型无法正确回答这类问题，因为它们基于互联网的某一时间点的静态快照。
- RALMs虽然可以通过更新文档索引来更好地处理这些问题，但在科学、医疗和法律出版物等领域，简单的文档替换并不足以解决问题，因为这些领域的更新通常涉及添加新文档而非替换现有文档。

TempRALM：

- 本研究在Atlas的基础上提出了TempRALM，一个具有少量样本学习能力的新RALM，能够处理包含多个版本文档的文档索引，无需重训练模型。

3 方法论

3.1 RALM架构概述

- RALM架构包含三个核心元素：文档索引、检索器和生成器（Reader）。
- 文档索引包含文档，检索器负责识别给定查询相关的文档，生成器（Reader）基于检索到的文档和查询生成回答。

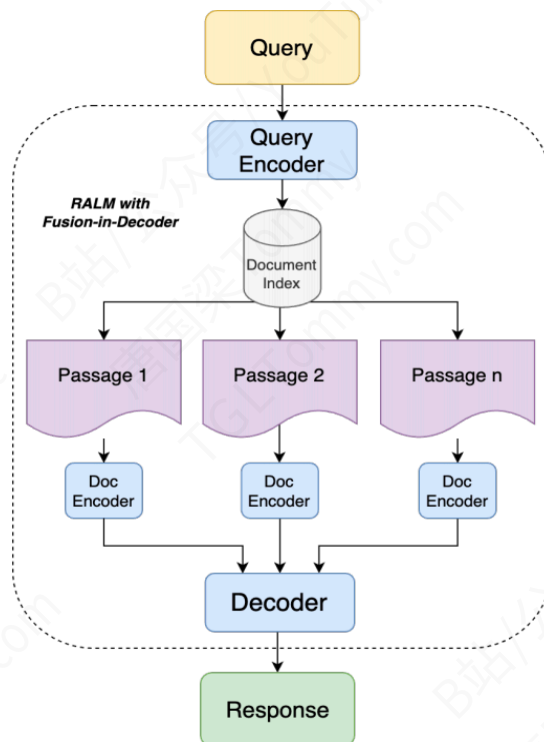


Figure 2: Retrieval Augmented Language Model with Fusion-in-Decoder Architecture

3.2 语义检索得分

- 决定哪些文档与查询相关的核心计算基于计算查询和每个文档之间的语义相似度得分。
- 该得分是通过编码器将查询和文档独立编码后的编码器表示的点积计算出来的。

方法论详述：

- 本研究在Atlas模型基础上构建，引入了时间检索方法，并评估了所得模型的性能。
- 使用预训练的Atlas生成器和检索器组件，检索器基于Contriever信息检索方法，生成器基于T5架构。
- 这些组件结合了Fusion-in-Decoder修改和语言建模适配。
- 本文提出的方法可以扩展任何通过查询和文档语料库之间的数值相似度得分来排名文档的检索器。

Contriever: <https://github.com/facebookresearch/contriever.git>

3.3 时间得分

时间得分介绍：

- TempRALM方法增加了一个新的时间得分 $\tau(qt, dt)$ ，其中 qt 是查询时间， dt 是文档时间。
- 时间得分 τ 的计算取决于 qt 和 dt 的时间差的倒数，时间差越小，得分越高。
- 使用缩放因子 α 来调整时间得分的重要性，与语义得分的重要性相对应。

时间得分公式：

$$\tau(qt, dt) = \frac{\alpha_{scale}}{qt - dt}$$

时间得分与语义得分的对齐：

- 为了将时间得分与语义得分 $s(q, d)$ 对齐，执行了标准化过程。
- 从每个时间得分中减去所有 q, d 对的时间得分平均值 μ_τ ，然后除以标准差 σ_τ 。
- 标准化后的时间得分随后按照语义得分的数值范围进行缩放，方法是乘以语义得分的标准差 σ_s 并加上语义得分的平均值 μ_s 。

标准化时间得分公式：

$$\tau(qt, dt) = \left(\frac{\tau(qt, dt) - \mu_\tau}{\sigma_\tau} \right) \times \sigma_s + \mu_s$$

该公式确保时间得分与语义得分在数值上是可比的，允许两者结合以产生最终的相关性得分。通过这种方法，TempRALM模型能够在检索文档时考虑到查询的时间相关性，从而提高检索结果的准确性。

>>> 时间得分：举例讲解 <<<

时间得分的目的是评估查询时间和文档时间的接近程度，从而确保检索到的文档不仅在内容上与查询相关，而且在时间上也是相关的。

1. 基本公式：

时间得分的基本公式是 $\tau(qt, dt) = \frac{\alpha_{scale}}{qt - dt}$ 。

2. 其中， qt 是查询的时间戳， dt 是文档的时间戳， α_{scale} 是一个缩放因子，用于调整时间得分的相对重要性。

这个公式的核心是时间差的倒数。如果查询时间与文档时间非常接近，它们的差值 $qt - dt$ 会很小，那么其倒数就会很大，得到的时间得分就会很高。相反，如果时间差很大，那么得分会低。

3. 标准化：

由于不同的 q, d 对可能产生不同范围的时间得分，我们需要对这些得分进行标准化，以便它们能够与语义得分结合在一起使用。

$$\text{公式 } \tau(qt, dt) = \left(\frac{\tau(qt, dt) - \mu_\tau}{\sigma_\tau} \right) \times \sigma_s + \mu_s$$

是标准化过程，其中 μ_τ 是所有时间得分的平均值， σ_τ 是标准差。通过减去平均值并除以标准差，我们得到一个标准化的分数，然后通过乘以语义得分的标准差 σ_s 并加上平均值 μ_s 来调整其数值范围。

例子：

假设我们有三个文档和一个查询，查询时间 (qt) 是2024年1月1日。我们的三个文档分别有以下时间戳 (dt)：

- 文档1: 2023年12月31日
- 文档2: 2022年1月1日
- 文档3: 2024年1月1日

我们也假设 α_{scale} 是1（简化计算）。

1. 计算时间得分：

- 对于文档1， $qt - dt$ 是1天，所以 $\tau(qt, dt)$ 是 $\frac{1}{1}$ 。
- 对于文档2， $qt - dt$ 是365天，所以 $\tau(qt, dt)$ 是 $\frac{1}{365}$ 。

- 对于文档3, qt 和 dt 是相同的, 为了避免除以零, 我们可以假设当天的文档有一个非常小的时间差, 例如0.01天, 所以 $\tau(qt, dt)$ 是 $\frac{1}{0.01}$ 。

2. 标准化时间得分:

假设这三个时间得分的平均值 μ_τ 是某个数字, 标准差 σ_τ 也是某个数字。我们将每个文档的时间得分减去平均值然后除以标准差, 得到标准化的得分。

这样, 我们就能得到每个文档的时间得分, 可以与语义得分结合, 从而计算出每个文档相对于查询的综合得分。综合得分最高的文档将被认为是对于特定查询最相关的文档。通过这种方式, TempRALM模型在检索时不仅考虑内容的相关性, 还考虑了时间的相关性。

3.4 TempRALM: 一个时间增强型检索器

① TempRet得分:

- TempRet是一个辅助检索得分函数, 结合语义得分 $s(q, d)$ 和时间得分 $\tau(qt, dt)$ 。
- 得分公式为 $TempRet(q, d, qt, dt) = s(q, d) + \tau(qt, dt)$ 。

② 检索器的扩展:

- 扩展检索器以排除与查询时间未来相关的信息。
- 如果文档时间戳大于查询时间戳, 将检索得分设为负无穷 ($-\infty$), 这样在排名时就不会考虑这些文档。

③ 得分计算公式:

- $TempRet(q, d, qt, dt)$ 的计算方式如下:

$$TempRet(q, d, qt, dt) = \begin{cases} s(q, d) + \tau(qt, dt) & \text{if } qt \geq dt \\ -\infty & \text{otherwise} \end{cases} \quad (5)$$

- 这确保了只考虑时间上在查询时间戳之前或相等的文档。

④ 为确保广泛覆盖:

- 从检索器中获取过多的文档集, 并从这个集合中选择得分最高的 $topk$ 文档, 作为语言模型的输入。
- 通过实验找到最佳的过度检索文档数量。
- 重要的是, 这些扩展不需要重新训练RALM的任何部分。

3.5 评估指标

评估方法:

- 评估系统在特定时间的问答测试集上的有效性。
- 假设有一个包含查询、时间戳和答案三元组的少量训练集

$$Q_{train} = \{(q_1, qt_1, a_1), (q_2, qt_2, a_2), \dots, (q_N, qt_N, a_N)\}。$$

- 在保留出的查询、时间戳和答案测试集上进行评估。
- 评估指标是测试集中每个数据点的预测答案与真实答案之间的精确匹配度。

4 实验设置

4.1 TempRALM组件配置

- TempRALM检索器使用标准的Atlas-large配置，并增加了时间扩展。
- 它基于一个通用的基于对比器的密集检索器和一个建立在Fusion-in-Decoder架构上的序列到序列模型，使用T5-1.1进行语言建模适配。
- 检索器和生成器使用与Atlas相同的预训练来初始化。

实验环境：

- 实验在单个NVIDIA A100-PCIE-40GB GPU上进行。

4.2 文档索引生成

- 开始时使用维基百科生成文档索引，但由于页面结构和额外文本导致模型性能不佳，转而使用ATP/WTAT网球单打比赛数据表。
- 数据表包含比维基百科更详细的信息，如选手、比赛日期和地点。
- 实验从ATP表中筛选出四大满贯赛事，将表中的每行数据转换为自然语言描述，包括赛事名称、胜者、比赛日期和最终比分。
- 提取了日期时间戳（格式为YYYY-MM-DD），并将其作为元数据附加到段落中。
- 跨越1978至2019年，共生成了50,000份文档。

实验参数选择：

- 为TempRALM和Atlas-large选择参数，包括各种超参数如每个问题检索的topk文档数、训练步数、检索器和语言模型学习率及采样温度。

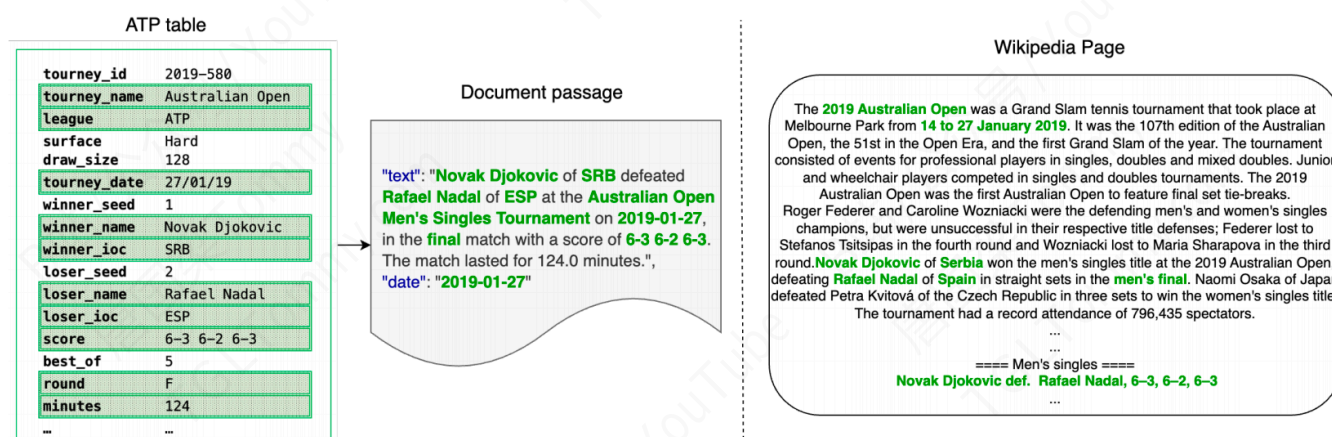


Figure 3: Example of tabular data to passage conversion. We convert relevant features of the tabular dataset to textual passages.

4.3 训练和评估数据

输入数据格式：

- 使用的数据格式为：(query, timestamp, answer)，时间戳表示查询提出的时间，答案对应于提出查询时有效的答案。

例子：

- 以查询“2012年5月14日谁赢得了澳大利亚公开赛女单决赛？”为例，正确的答案应该是截至2012年5月14日的澳大利亚公开赛女单冠军。
- 一旦新的锦标赛结束，相同查询的回应将发生变化。

4.3.1 少量样本训练集

- 创建了3个大小分别为32，64和128个数据点的非重叠少量样本训练集。
- 每个数据点都包含(query, timestamp, answer)格式，以确保查询类型分布均衡，包括关于冠军、亚军、决赛选手以及最终比分的查询，涵盖1978至2018年的四大网球锦标赛。
- 根据Atlas作者的建议选择少量学习训练集的大小，他们发现64个训练实例通常能得到合理的性能。

4.3.2 测试集

- 创建了两个大小为128的评估测试集，每个测试集都使用与训练集相同的格式构建真值三元组，但专注于2019年的锦标赛数据，确保训练集和测试集之间有明确的分隔。
- 这两个测试集被称为时间接近性查询集（TPQs），分别是TPQ-2019和TPQ-2020。两个查询集包含相同的问题，涉及2019年的比赛，但时间戳分别来自连续的两年，2019年和2020年。
- 例如，针对“谁赢得了2019年美国公开赛女单决赛？”的问题，TPQ-2019将时间戳标记为2019年（例如12-31-2019），而TPQ-2020将时间戳标记为2020年（例如01-01-2020）。在两种情况下，答案都是指2019年9月举行的美国公开赛。

正确回答问题需要模型能够超越文本模式匹配，具有（或能够模拟）确定时间接近性的时间意识。文中提到的图4展示了如何构建与特定段落相关的查询。

Query	Timestamp	Answer	Corresponding Passage
Who won the Australian Open women's Singles final	03/10/19	Naomi Osaka	Naomi Osaka of JPN defeated Petra Kvitova of CZE at the Australian Open Women's Singles Tournament on 2019-01-14, in the final match with a score of 7-6(2) 5-7 6-4. The match lasted for 147.0 minutes.
Who was runner-up in the Australian Open women's Singles final	10/15/19	Petra Kvitova	
Who played in the Australian Open Women's Singles final?	01/01/20	Naomi Osaka and Petra Kvitova	
What was the score in Australian Open Women's Singles final?	01/01/20	7-6(2) 5-7 6-4	

Figure 4: Questions asking about the winner, runner-up, finalists and score asked at different timestamps, where the right answer corresponds to the most recent tennis match that occurred before the query timestamp

5 结果

基线实验

- 比较了TempRALM与Atlas-large在TPQ-2019和TPQ-2020测试集上的性能，测试集基于32、64和128个样本的少量学习场景。
- 使用精确匹配指标，并评估了两种模型的检索器性能，使用recall@1和recall@5指标。

5.1 TempRALM与Atlas-large的比较

- 当查询的时间戳与事件匹配时（TPQ-2019），Atlas表现与TempRALM相当。
- 当时间戳不匹配时（TPQ-2020），TempRALM在所有少量学习实验中均显著优于Atlas-large，分别提高了49%、67%和74%。
- 这显示了时间增强在回答生成中的关键作用，特别是当查询和段落的时间不再匹配时。
- 随着训练样本数量的增加，时间增强的影响逐渐增大。
- 发现少量学习环境下，当样本数达到64时，模型性能开始变得合理，并且在128个训练样本时性能提高缓慢。

Train Set	TPQ-2019 Test Set Exact Match		TPQ-2020 Test Set Exact Match	
	Atlas	TempRALM	Atlas	TempRALM
32	64.84	71.72	37.65	55.93
64	74.68	75.78	40.93	68.44
128	76.88	77.80	41.72	72.65

表1：基于时间标记匹配的时间接近性与时间的比较：我们评估了两个查询集的检索性能——TPQ-2019和TPQ-2020，这两个查询集包含关于一个在2019年发生的事件的相同问题，但时间戳是连续两年的，用于模拟查询的时间范围。该表展示了Atlas-large模型和TempRALM之间的性能比较。这两个模型都是在包含32、64和128个带有时间后缀的问答对组成的有限训练集上进行微调的。一个健壮模型应该能够根据查询与相关事件相关段落的时间接近性来准确回答查询，而不仅仅依赖于查询和段落中匹配的年份标记。我们的模型在两个查询集中都表现出强大的性能。

5.2 检索器性能分析

- 观察到TempRALM成功检索到正确的段落，但LLM生成的答案错误，表明问题在LLM。
- TempRALM和Atlas-large的recall@1和recall@5指标都有所提高，显示时间检索改进了这两项指标。
- 在TPQ-2020实验中，TempRALM相比Atlas-large的recall@1提高了165%，这突显了Atlas-large检索器处理时间数据的不足和TempRALM在此方面的显著能力。

	TPQ-2019		TPQ-2020	
Model	Recall@1	Recall@5	Recall@1	Recall@5
Atlas-Large	0.54	0.63	0.24	0.63
TempRALM	0.63	0.75	0.64	0.75

表2：检索器召回指标：我们计算在64-shot训练示例设置中的召回率。我们选择一个实验，该实验最接近我们所有64-shot Atlas和TempRALM实验的平均精确匹配。

5.3 模型大小性能：大型与基础模型的比较

性能评估：

- 在Atlas large模型（阅读器770M参数/检索器110M参数）上进行了性能评估。
- 同时比较了基础模型（阅读器220M参数/检索器110M参数）的性能，发现时间检索在不同模型大小上都表现更好。

Model Size	Test Set Exact Match	
	TPQ-2019	TPQ-2020
Atlas Base	42.97	28.13
TempRALM Base	56.25	35.94
Atlas Large	74.68	40.93
TempRALM Large	75.78	68.44

表3：模型大小性能比较：我们比较了基础模型和大型模型在64-shot训练示例设置中Atlas和TempRALM之间的精确匹配情况。

性能分析：

- TempRALM在基础和大型模型中都显示出了优越的性能，尤其是在TPQ-2020测试集中，TempRALM Large与Atlas Large相比，精确匹配得分提高了明显。
- 这表明TempRALM模型不仅在大型模型中有效，而且在较小的模型中也能够有效地处理时间相关的查询。
- 在TPQ-2019和TPQ-2020测试集上，TempRALM模型与Atlas模型相比，都显示出了性能上的改善，特别是在处理时间戳不匹配的TPQ-2020测试集时，TempRALM Large模型的性能提升尤为显著。

6 结论和未来工作

结论：

- 本研究介绍并评估了TempRALM，这是一个增加了时间感知的检索增强语言模型（RALM）。
- 与传统依赖于语义相似性的RALM方法不同，TempRALM同时考虑了语义和时间相关性来选择文档，以响应给定的查询。
- 结果显示，与Atlas-large模型相比，TempRALM的性能提升了高达74%，即使文档索引中存在多个版本的文档（来自不同时间点）。
- 值得注意的是，这一成就无需模型预训练、替换文档索引，或添加任何其他计算密集的元素。

未来工作：

- 计划基于本文提出的工作探索多种途径，包括为时间相关性函数的参数实施和评估不同的学习策略，以及探索检索器与LLM之间的相互作用。
- 此外，计划将时间检索方法应用于其他任务，如事实核查、推荐系统和检索增强的对话代理。